1. Normalization (Deep Understanding)

1. Definition: Structuring DB to remove redundancy and improve integrity.
2. Why: Prevent anomalies (insert, update, delete).
3. Better because: Ensures consistency across system.
4. Where used: Banking, transaction systems.
5. Real-world: User info stored once, referenced everywhere.

2. Denormalization

1. Definition: Adding redundancy to improve read performance.
2. Why: Reduce JOIN complexity.
3. Better because: Faster queries for large datasets.
4. Where used: Dashboards, analytics.
5. Real-world: Order history storing product name.

3. Indexing Advanced

1. Types: B-Tree (default), Hash (fast lookup).
2. Why: Avoid full table scans.
3. Better: Improves query speed drastically.
4. Where: Frequently searched columns.
5. Example: email index in users table.

4. SQL JOINs (Complete)

1. INNER JOIN: Only matching rows.
2. LEFT JOIN: All left + matched right.
3. RIGHT JOIN: All right + matched left.
4. FULL JOIN: All records from both.
5. Why: Combine related data.
6. Example: orders + users → customer details.

5. Query Optimization Advanced

1. Use indexes effectively.
2. Avoid SELECT *.
3. Analyze execution plans.
4. Reduce joins where possible.
5. Real-world: Optimize slow dashboard queries.

6. Transactions & ACID

1. Atomicity: All or nothing.
2. Consistency: Valid state.

3 Isolation: Independent execution.
4 Durability: Permanent storage.
5 Example: Bank transfer.

7. Concurrency Control

1 Problems: Dirty reads, deadlocks.
2 Solutions: Locks, isolation levels.
3 Why: Multiple users access DB simultaneously.
4 Example: Ticket booking system.

8. Scaling Databases

1 Vertical scaling: Increase machine power.
2 Horizontal scaling: Add more machines.
3 Sharding: Split data.
4 Replication: Duplicate DB.
5 Example: Millions of users in apps.

9. ORM vs Raw SQL

1 ORM: Easier development.
2 Raw SQL: More control.
3 Why ORM: Faster development.
4 Example: SQLAlchemy mapping tables to classes.

10. System Design Integration

1 Flow: Client → API → DB → Response.
2 Use caching (Redis) for performance.
3 Use microservices for scalability.
4 Example: E-commerce architecture.